From Optimal Placement to Team Tracking on Patrol Roadmaps in Polygons

Shashwata Mandal[1] and Sourabh Bhattacharya[1]

***Abstract*—In this work, we address the deployment and motion planning problem for mobile pursuers confined to a patrol roadmap while tracking mobile evaders inside a known, simply-connected polygon. An evader can break visual contact only by crossing into the shadow of a reflex corner. Every reflex corner has an *association region* from which it casts no shadow, and occupying this region before the evader reaches the corner is a necessary condition to prevent escape there. First, we construct a patrol roadmap from Keil-derived kernel candidate points by solving an exact $0/1$ integer program, so that every association region is intersected by the roadmap. Next, we score the timing race at each corner by the ratio α of the pursuer's roadmap distance to the association region over the evader's geodesic distance to the corner. Per-frame optimal guard placement is then a dynamically weighted network 1-center problem, which we solve exactly by an upper-envelope routine. A stable-node roadmap controller turns placement into tracking, and an ε-slack layer spends near-optimal timing margin on line of sight. We then characterize the limit of a single pursuer through the critical speed ratio $s^\star$: across nine benchmark polygons, a pursuer requires 1.42–$2.71\times$ the evader's speed to satisfy the necessary timing condition everywhere. Finally, we extend the framework to pursuer teams through a mixed-integer program that partitions the corners, minimizing the team's required speed ratio and then maximizing persistent line of sight. Multiple evaders are handled by a nearest-evader rule. We validate the framework in simulation against five baselines and three evader models, and in closed-loop tracking on a simulated hexacopter in ROS 2 and Gazebo.**

***Index Terms*—Pursuit-evasion, visibility, roadmap planning, guard placement, polygon decomposition, target tracking.**

I. Introduction

Autonomous mobile robots and unmanned aerial vehicles (UAVs) are increasingly deployed for surveillance, security patrol, and search-and-rescue [1]–[4]. A central task in these missions is maintaining visual contact with mobile intruders (evaders) among obstacles. In many deployments, the geometry is known in advance, the mobile guards (pursuers) are few, and they travel along a precomputed patrol roadmap rather than freely through the workspace. In this work, we investigate this setting: one or more pursuers constrained to a patrol roadmap inside a known, simply-connected polygon, tracking evaders that move freely and attempt to break line of sight.

In a polygon, an evader can break visual contact only by crossing behind a *reflex vertex* (or *corner*), where the interior angle exceeds π. Maintaining uninterrupted visibility of a mobile evader among obstacles is intractable in general [5], and a pursuer confined to a roadmap inherits every such limit. The *timing race* at each corner, however, can be analyzed exactly. Each reflex corner c has an *association region* $\mathcal{A}(c)$, the set of positions from which c casts no shadow. If a pursuer occupies $\mathcal{A}(c)$ before the evader reaches c, the evader cannot use c to break line of sight. Conversely, reaching $\mathcal{A}(c)$ in time is a *necessary* condition to deny an escape around c. The race is scored by the ratio

$$\alpha_c \;=\; \frac{d_{\mathcal{G}}(p,\, \mathcal{A}(c))}{d_{\text{geo}}(e,\, c)}, \tag{1}$$

of the pursuer's shortest-path distance along the roadmap $\mathcal{G}$ to $\mathcal{A}(c)$ over the evader's geodesic distance to c. The pursuer wins the race whenever $\alpha_c \le s_p/s_e$, where s_p and s_e are the maximum speeds of the pursuer and the evader.

To build a patrol infrastructure for this race, we draw on kernel-based coverage [6]–[8]. Non-convex polygons rarely have a non-empty kernel, so we use Keil's reflex-chord construction to generate a finite pool of kernel candidate points (KER points), and an exact $0/1$ integer linear program (ILP) selects a minimal guard subset such that every association region contains a guard. Connecting the guards by geodesic shortest paths yields a patrol roadmap $\mathcal{G}$ on which every association-region distance is finite.

On this roadmap, we develop a staged framework from placement to closed-loop tracking. First, we find the roadmap point g^* that minimizes the worst-case ratio over all corners, $\min_{q\in\mathcal{G}} \max_c \alpha_c(q, e)$. This is a dynamically weighted continuous 1-center problem on a network. Along each roadmap edge the per-corner ratios are piecewise-linear tent functions, and the lowest point of their upper envelope is found exactly by a slab–duality routine. Second, we turn per-frame placement into tracking by driving the pursuer toward g^* with a stable-node roadmap controller that removes replanning oscillations. An ε-slack visibility layer chooses, among near-optimal positions, the one that sees the evader. Third, we introduce the *critical speed ratio* $s^\star$, the speed ratio a single pursuer needs to satisfy the necessary condition from every evader position. Because $s^\star > 1.4$ on all nine benchmark environments, a single pursuer at or below the evader's speed cannot cover every corner. Finally, since α is separable across corners, we extend the framework to k pursuers by partitioning the corners. An exact mixed-integer linear program (MILP) minimizes the team's critical speed ratio $s_k^\star$ and then maximizes persistent line of sight along escape paths. Multiple evaders are handled by a nearest-evader rule.

The main contributions of this work are as follows:

(i) A **patrol roadmap** built from a minimal subset of Keil-derived KER points selected by exact set cover, so that

Manuscript received: Month, Day, Year; Revised Month, Day, Year; Accepted Month, Day, Year.

[1]The authors are with Iowa State University, Ames, IA 50011, USA. {smandal, sbhattac}@iastate.edu

every association region contains a patrol node, and a **necessity lemma** linking corner timing races to visibility (Sections III and IV).

(ii) **Exact per-frame guard placement** g^* as a dynamically weighted network 1-center problem, with a **stable-node roadmap controller** and an **ε-slack visibility layer** (Sections V–VI).

(iii) The **critical speed ratio** $s^\star$, a necessary speed bound for single-pursuer cutoff, measured on nine environments and tested against an escaping adversary (Section VIII-E).

(iv) A **multi-pursuer and multi-evader extension** that partitions the corners by an exact MILP over team speed ratio and persistent visibility along escape paths (Section VII).

(v) **Simulations** on nine polygons against five baselines and three evader models, and a **closed-loop ROS 2/Gazebo deployment** on a hexacopter (Section VIII).

II. Related Work

Chung et al. [4] survey search and pursuit-evasion in mobile robotics, and Robin and Lacroix [9] review multi-robot target tracking. We discuss the work closest to ours.

A. Pursuit-Evasion in Polygonal Environments

Continuous pursuit-evasion traces back to the lion-and-man problem [10] and to differential games [11]. In polygons with obstacles, Bhadauria et al. [12] showed that three pursuers suffice to capture an equal-speed evader under full visibility. Noori and Isler [13] showed that a single lion restricted to line-of-sight visibility still captures the man in monotone polygons. Parsons [2] introduced *graph searching* and Suzuki and Yamashita [3] the corresponding *polygon search* problem. LaValle et al. [1] and Guibas et al. [14] established visibility-based pursuit-evasion in polygons, where some simply-connected polygons require $\Theta(\log n)$ pursuers for detection. Later work covers limited fields of view [15], randomized strategies [16], complete algorithms [17], and surveillance graphs [18]. These works concern *search* for an unseen target. We study *tracking* of a detected one.

B. Target Tracking with Visibility Constraints

Maintaining line of sight to a mobile evader among obstacles is the problem closest to ours. Murrieta-Cid et al. [5] showed that deciding whether a strategy maintains visibility is NP-complete in general, and Bhattacharya and Hutchinson [19] showed that even a free planar pursuer prevents escape only from a bounded set of initial configurations. Around a single reflex corner the game is completely decidable: escape and tracking regions as a function of the speed ratio [20], equilibrium strategies [21], and optimal pursuit trajectories [22] are known. Related analyses cover corner partitions in general polygons [23], circular obstacles [24], [25], and finite sensing range [26], [27]. Multi-robot variants maximize collective target visibility [28], [29] or allocate targets across observers [30], and a sufficient speed advantage always admits a search strategy [31]. A pursuer confined to a roadmap has a strictly smaller strategy set than a free pursuer, so these hardness results carry over, which is why we analyze the *timing race* to each corner rather than attempt a visibility guarantee.

On the reactive side, González-Baños et al. [32] steered a free observer by the target's shortest distance to an occlusion boundary, and Bandyopadhyay et al. [33] introduced *vantage time*, which combines the travel times of robot and target. Our ratio α is in this lineage, but it is roadmap-constrained, taken in the worst case over all corners, and minimized exactly over the network at every frame. Occlusion-aware aerial tracking [34]–[36] and learned visibility-based pursuit [37] optimize local visibility without a global corner-cutoff bound.

C. Guard Placement and Art Gallery Problems

The art gallery problem [38]–[40] asks for the minimum number of static guards that see an entire polygon. Minimizing the number of guards is NP-hard [41] and $\exists\mathbb{R}$-complete for point guards [42]. Practical exact solvers discretize guard and witness candidates and solve the resulting set cover by integer programming [43], [44]. We adopt the same methodology at the scale of reflex corners, with a different requirement: rather than covering area, every corner's association region must contain a patrol guard, so that the timing race to that corner is well posed. Our candidates come from star-shaped decomposition [45], [46]. Keil [6], [47] showed that extending chords through reflex vertices partitions a polygon into star-shaped pieces and identifies a finite set of points that anchor their kernels. We do not decompose the polygon but take these reflex-chord intersections as a finite pool of *KER candidate points* and solve a set-cover ILP for a minimal guard set.

D. Roadmap-Constrained Motion and Surveillance

Probabilistic roadmaps [48], visibility-based PRMs [49], and medial-axis or Voronoi skeletons [50] are built for collision-free navigation. Our patrol roadmap is instead a *surveillance infrastructure* whose vertices are guard positions derived from the polygon's reflex geometry. Graph patrolling [51], [52] and persistent monitoring [53], [54] optimize visitation frequency or latency along fixed tours rather than reacting to an evader. Isler et al. [55] restrict both players to a shared roadmap, and Becerra et al. [56] confine the evader to a reduced visibility graph while the observer moves freely. Here only the pursuer is confined.

Closest to our work, Laguna and Bhattacharya [57] studied guards on polygon diagonals under a distance-ratio speed condition of the same form as (1), showed that minimizing the required guard speed is NP-hard, and later gave fixed observer routes with bounded speed approximation [58] and an iterative roadmap construction for tracking [59]. Our earlier work partitioned polygons into star-shaped regions for relay tracking and coverage [7], [60] and introduced a kernel-based control law for scalable surveillance [8]. Those approaches bound a *sufficient* observer speed along a route or apply reactive velocity fields. The present work solves for the exact min–max guard position at every frame, characterizes the *necessary* speed bound $s^\star$, and partitions the corners among a team.

III. Problem Formulation

We consider a mobile agent, called the *observer*, equipped with a camera in a bounded environment. The environment is modeled as a simply-connected polygon $\mathcal{P} \subset \mathbb{R}^2$ (Fig. 1). The observer tries to maintain a line-of-sight (LOS)[1] with another mobile agent, called the *intruder*, located inside the polygon. Both agents are assumed to be point-sized holonomic agents on the plane with maximum speeds $s_p > 0$ (observer) and $s_e > 0$ (intruder). We assume that the observer has no knowledge about the current and future actions of the intruder. However, both agents are assumed to have complete information about the environment, and their locations in the environment.

Let $v_0, v_1, \ldots, v_{n-1}$ denote the list of vertices that form boundary of the polygon in a counter-clockwise order. Let $\mathcal{C} \subset \{0, \ldots, n-1\}$ denote the index set of the reflex vertices[2] in the environment. In this paper, the terms *reflex vertex* and *corner* are used interchangeably. For two points $a, b \in \mathcal{P}$, the *geodesic distance* $d_{\text{geo}}(a, b)$ between them is defined as the length of the shortest path from a to b that stays inside $\mathcal{P}$. In a simple polygon, this path is a polyline that bends only at reflex vertices. We compute geodesic distances on a precomputed visibility graph [61].

In this work, we assume that the observer is equipped with an omnidirectional camera, and its range is greater than the diameter of the polygon. Two points, a and b, are said to be *visible to each other* if $\overline{ab}$ (the segment joining them) lies inside $\mathcal{P}$. The *visibility polygon* of a point $a \in \mathcal{P}$ (denoted as $V(a)$) is defined the set of points in $\mathcal{P}$ that sees a. Since the polygon is simply-connected, loss of visibility is attributable to some corner inside the polygon. For a corner $c = v_i$ (Fig. 1),

$$\hat{u}_1 = \frac{v_i - v_{i-1}}{\|v_i - v_{i-1}\|}, \qquad \hat{u}_2 = \frac{v_i - v_{i+1}}{\|v_i - v_{i+1}\|} \tag{2}$$

are the unit vectors pointing inside $\mathcal{P}$ along the edges incident to v_i. Let $Cone(v_i)$ denote the region inside the convex cone [3] generated by $\hat{u}_1$ and $\hat{u}_2$ with v_i as the origin that lies inside the polygon. The *association region* of v_i,

$$\mathcal{A}(v_i) \;=\; V(v_i) \;\cap\; Cone(v_i) \tag{3}$$

is the region inside $Cone(v_i)$ that is visible from v_i.

Let c be a reflex vertex of a simple polygon $\mathcal{P}$ and let $q \in \mathcal{P}$. (i) If $q \in \mathcal{A}(c)$, no point of $\mathcal{P}$ is occluded from q with c as the occluding vertex. (ii) If $q \notin \mathcal{A}(c)$ and a target is at c, the shadow of c with respect to q is a cone of nonzero aperture in $\mathcal{P}$. The target can then break line of sight from q by an arbitrarily small displacement.

A roadmap $\mathcal{G} = (V, E)$ is an undirected graph embedded in $\mathcal{P}$ whose edges are straight segments weighted by Euclidean length (thick gray segments in Fig. 1). For any two points q, q' of $\mathcal{G}$, $d_{\mathcal{G}}(q, q')$ is the shortest-path distance along $\mathcal{G}$, and the roadmap distance to a region $X \subseteq \mathcal{P}$ is

$$d_{\mathcal{G}}(p, X) \;=\; \min_{q \in \mathcal{G} \cap X} d_{\mathcal{G}}(p, q). \tag{4}$$

[1]There is an LOS between two points inside the polygon is the line segment joining them lies completely inside the polygon

[2]A vertex is *reflex* (or *concave*) if its interior angle exceeds π.

[3]Define a convex cone of 2 vectors

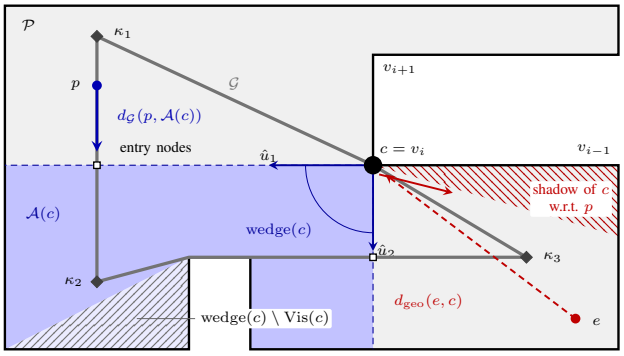


Fig. 1. Geometric terms on a polygon $\mathcal{P}$. The reflex corner $c = v_i$ has neighbors v_{i-1}, v_{i+1}, and the edge extensions $\hat{u}_1, \hat{u}_2$ span wedge(c) (dashed rays). The association region $\mathcal{A}(c) = \text{Vis}(c) \cap \text{wedge}(c)$ (dark blue) excludes the gray-hatched triangle occluded by the pillar. Thick gray segments form the patrol roadmap $\mathcal{G}$ connecting guards $\kappa_1, \kappa_2, \kappa_3$, with hollow squares marking entry nodes into $\mathcal{A}(c)$. Because $p \notin \mathcal{A}(c)$, corner c casts a shadow with respect to p (red hatch, Lemma **??**).

A. The α Timing Criterion

For a pursuer at $p \in \mathcal{G}$, an evader at $e \in \mathcal{P}$, and a corner c, we define

$$\alpha_c(p, e) \;=\; \frac{d_{\mathcal{G}}(p,\, \mathcal{A}(c))}{d_{\text{geo}}(e,\, c)}. \tag{5}$$

This ratio scores the *timing race to corner* c of Fig. 1. The evader reaches c no earlier than $d_{\text{geo}}(e, c)/s_e$, and the pursuer enters $\mathcal{A}(c)$ no earlier than $d_{\mathcal{G}}(p, \mathcal{A}(c))/s_p$, so the pursuer can occupy $\mathcal{A}(c)$ first if and only if $\alpha_c(p, e) \le s_p/s_e$. If instead $s_p/s_e < \alpha_c(p, e)$, the evader reaches c while $p \notin \mathcal{A}(c)$ and, by Lemma **??**(ii), can break line of sight there. Winning every corner race,

$$\max_{c \in \mathcal{C}} \alpha_c(p, e) \;\le\; s_p/s_e, \tag{6}$$

is therefore *necessary* for uninterrupted visibility. It is not sufficient: a pursuer that wins every race may still lose line of sight if the evader crosses an occlusion ray far from the corner, or follows a non-geodesic path [62]. The ratio α is thus an exact characterization of the worst-case *timing precondition* for cutoff, which we complement with line-of-sight objectives in Sections V-D and VII.

B. Optimization Objectives

From (5) we define two quantities. For an evader position e, the instantaneous *optimal guard position* is the roadmap point that minimizes the worst-case ratio over all corners,

$$g^*(e) \;=\; \arg\min_{q \in \mathcal{G}} \; \max_{c \in \mathcal{C}} \; \alpha_c(q, e), \tag{7}$$

with value $\alpha^*(e) = \max_{c \in \mathcal{C}} \alpha_c(g^*(e), e)$. Taking the worst case over evader positions gives the *critical speed ratio*

$$s^\star \;=\; \sup_{e \in \mathcal{P}} \alpha^*(e) \;=\; \sup_{e \in \mathcal{P}} \min_{q \in \mathcal{G}} \max_{c \in \mathcal{C}} \alpha_c(q, e), \tag{8}$$

which by (6) is the least speed ratio at which a single roadmap-constrained pursuer can satisfy the necessary timing condition from every evader position. Three caveats apply to $s^\star$. It is necessary but not sufficient, since winning the races does not prevent occlusion-ray crossings away from the corner. It is an

oracle bound, since it credits the pursuer with already standing at $g^*(e)$, and Section VIII-I measures the transit gap a moving pursuer experiences relative to this oracle. And we evaluate the supremum on a uniform grid of interior evader positions, which gives a lower bound on the continuous supremum.

In online tracking, the pursuer computes $g^*(e(t))$ at each frame and moves toward it along $\mathcal{G}$, staying on the roadmap and suppressing replanning oscillations caused by target motion.

C. Assumptions

The workspace $\mathcal{P}$ is a known, simply-connected polygon, the roadmap $\mathcal{G}$ is precomputed offline, and both agents are points. The evader's policy is unknown, and the pursuer reacts to the current evader state at each step. Our main trials use the regime $s_p \le s_e$, in which the pursuer has no speed advantage, and Section VIII-E sweeps s_p/s_e across $s^\star$.

IV. Guard-Set Construction and Patrol Roadmap

In this section, we construct the patrol roadmap $\mathcal{G}$ from the polygon $\mathcal{P}$ in the five stages of Fig. 2. Two point sets are kept distinct throughout: the *KER candidate set* $\mathcal{K}$ generated by extending reflex chords, and the *guard set* $\mathcal{J} \subseteq \mathcal{K}$, the minimal cover selected from it to serve as patrol nodes.

A. Stages 1–2: Corner Detection and Boundary Contraction

Stage 1 extracts the reflex corners $\mathcal{C}$ with the cross-product test of Section III. A reflex corner often dominates the visibility of several adjacent boundary segments, so Stage 2 clusters the boundary vertices that jointly govern the same local region by the iterative *boundary contraction* of Algorithm 1.

The immediate neighbor of a corner is always visible along the boundary edge, so the vertex two steps away is the first informative test: a straight geodesic to it certifies that the intervening vertex causes no occlusion and can be merged into cluster$[c]$. Each accepted contraction removes one boundary vertex, so the loop terminates in at most n iterations, leaving each corner with a cluster of nearby vertices whose combined visibility describes the wall around it.

B. Stage 3: Association Regions

For each corner $c = v_i$, we compute $\mathrm{Vis}(c)$ and intersect it with $\mathrm{wedge}(c)$ to obtain $\mathcal{A}(c)$ of (3). The wedge is clipped as the triangle $\{v_i,\ v_i + R\hat{u}_1,\ v_i + R\hat{u}_2\}$ with $\hat{u}_1, \hat{u}_2$ from (2) and R larger than the diameter of $\mathcal{P}$. For guard selection only (Stage 4), coverage is tested against the subset of $\mathcal{A}(c)$ also

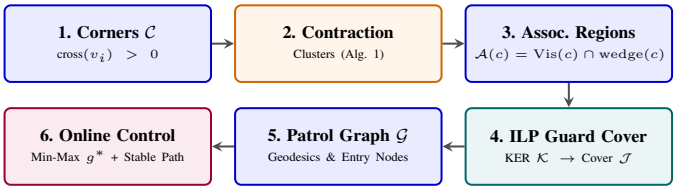


Fig. 2. Pipeline overview: from reflex corner detection and Keil's KER candidate pool $\mathcal{K}$, to exact $0/1$ ILP guard cover $\mathcal{J}$, geodesic patrol graph $\mathcal{G}$, and online α-optimal placement and tracking control.

Algorithm 1: Boundary Contraction

Input: Polygon vertex list boundary, concave corners $\mathcal{C}$
Output: Per-corner vertex clusters cluster$[c]$

```
while C ≠ ∅ do
    foreach corner c ∈ C, direction δ ∈ {−1, +1} do
        j ← index two steps from c along boundary in direction δ
        if geodesic path from c to v_j is a straight segment then
            cluster[c] += {v_j, v_{j+δ}}
            Remove v_{j+δ} from boundary
            break inner loop
    Remove corners from C that are no longer concave in the contracted boundary
```

visible from the vertices in cluster$[c]$, so that a selected guard sees the wall along which an evader may hide. The online α evaluation and the entry nodes (Stage 5) use $\mathcal{A}(c)$ itself, as Lemma **??** requires.

C. Stage 4: KER Candidate Set and Guard-Set Selection

a) KER candidate points.: Following Keil's decomposition [6], [47] and our kernel framework [8], for every corner c and each of its neighbors v' in the visibility graph we extend the chord cv' past c until it meets $\partial\mathcal{P}$. These chords bound the star-shaped pieces of Keil's decomposition and anchor their kernels, which makes their boundary hits and pairwise intersections natural guard candidates. The *KER candidate set* is

$$\mathcal{K} \;=\; \{v_i : i \in \mathcal{C}\} \;\cup\; \mathcal{K}_{\text{bnd}} \;\cup\; \mathcal{K}_{\text{int}}, \tag{9}$$

where $\mathcal{K}_{\text{bnd}}$ are the chord–boundary hits and $\mathcal{K}_{\text{int}}$ the chord–chord intersections.

b) Corner coverage map.: For each candidate $\kappa \in \mathcal{K}$, we determine the set $\mathcal{C}(\kappa) \subseteq \mathcal{C}$ of corners whose association regions contain κ, with a 1-unit boundary tolerance that absorbs fixed-point clipping error.

c) Exact guard-set selection.: Given the map $\kappa \mapsto \mathcal{C}(\kappa)$, selecting the fewest guards such that every association region contains one is the set-cover problem

$$\min_{\mathcal{J}\subseteq\mathcal{K}} |\mathcal{J}| \quad \text{s.t.} \quad \bigcup_{\kappa\in\mathcal{J}} \mathcal{C}(\kappa) \;=\; \mathcal{C}. \tag{10}$$

Set cover is NP-hard in general [63], but our instances have at most 29 corners and a few thousand candidates, so we solve (10) to optimality as the $0/1$ integer linear program

$$\min_{y\in\{0,1\}^{|\mathcal{K}|}} \sum_{\kappa\in\mathcal{K}} y_\kappa \qquad \text{s.t.} \sum_{\kappa\,:\,c\in\mathcal{C}(\kappa)} y_\kappa \;\ge\; 1, \quad \forall\, c \in \mathcal{C}, \tag{11}$$

with guard set $\mathcal{J} = \{\kappa \in \mathcal{K} : y^*_\kappa = 1\}$. Before invoking the solver, we remove every *dominated* candidate κ, one with $\mathcal{C}(\kappa) \subseteq \mathcal{C}(\kappa')$ for some $\kappa' \neq \kappa$: replacing κ by κ' never hurts, so pruning loses nothing, and it removes most

candidates on our polygons. The reduced ILP is solved with `scipy.optimize.milp` [64] (HiGHS), which returns a proven optimum in under one second on every benchmark and avoids the $\ln|\mathcal{C}|$ gap and tie-breaking sensitivity of greedy set cover [65].

D. Stage 5: Geodesic Patrol Graph

a) Edge generation.: We compute the geodesic shortest path inside $\mathcal{P}$ between every pair of guards in $\mathcal{J}$ on the visibility graph. Each straight segment of these paths becomes an edge of $\mathcal{G}$, with duplicates merged.

b) Entry nodes and distance tables.: For every corner c, we intersect the edges of $\mathcal{G}$ with the buffered association region $\mathcal{A}(c)$ and insert the intersection points as vertices, splitting their host edges. These *entry nodes* $Z_c \subset V$ make $\mathcal{G} \cap \mathcal{A}(c)$ a union of closed subsegments bounded by vertices of Z_c, so for any $q \in \mathcal{G}$ the distance $d_{\mathcal{G}}(q, \mathcal{A}(c))$ in (4) is attained at an entry node. Floyd–Warshall on $\mathcal{G}$ then gives the all-pairs distances D over V, and we store $D(u, \mathcal{A}(c)) = \min_{z \in Z_c} D(u, z)$ for every vertex u and corner c as the input to the per-frame placement algorithm. Buffering only enlarges $\mathcal{A}(c)$ and so only decreases α_c, so the necessity condition (6) and the bound $s^\star$ remain valid on the buffered regions. Because the set cover places a guard in every association region, requirement (**??**) holds for all $c \in \mathcal{C}$.

V. Computing the Optimal Guard Position

In this section, we present an exact algorithm for the optimal guard position $g^*(e)$ in (7) at each frame. Its inputs are the roadmap $\mathcal{G} = (V, E)$, the distance table $d_{\mathcal{G}}(u, \mathcal{A}(c))$ from every vertex $u \in V$ to every association region, and the evader's geodesic distances $L_c = d_{\text{geo}}(e, c)$ to the corners. It returns $g^*(e)$ and its value $\alpha^*(e)$, and runs edge by edge as illustrated in Fig. 3.

A. Per-Corner α Along a Roadmap Edge

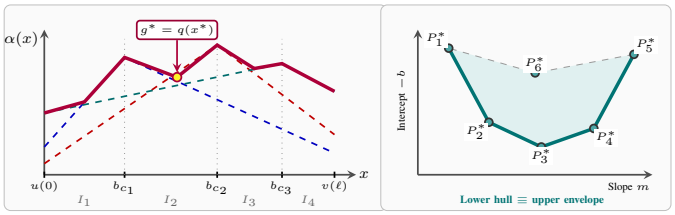
Consider an edge $uv \in E$ of length $\ell = d_{\mathcal{G}}(u, v)$ and let $x \in [0, \ell]$ parameterize the point $q(x)$ along it, with $q(0) = u$ and $q(\ell) = v$. For a corner c, let $d_u^c = d_{\mathcal{G}}(u, \mathcal{A}(c))$ and $d_v^c = d_{\mathcal{G}}(v, \mathcal{A}(c))$ be the tabulated distances from the endpoints. A pursuer at $q(x)$ reaches $\mathcal{A}(c)$ through u or through v, so

$$d_{\mathcal{G}}(q(x), \mathcal{A}(c)) \;=\; \min\big(d_u^c + x,\; d_v^c + \ell - x\big). \tag{12}$$

Dividing by L_c gives the per-corner ratio along the edge, $f_c(x) = d_{\mathcal{G}}(q(x), \mathcal{A}(c))/L_c$. The two branches of (12) meet at the *crossover point*

$$b_c \;=\; \frac{d_v^c + \ell - d_u^c}{2} \;\in\; [0, \ell], \tag{13}$$

so f_c is a *tent function* with slopes $\pm 1/L_c$ and apex at b_c (Fig. 3(a)).



(a) Primal space: $f_{c_1}, f_{c_2}, f_{c_3}$ and $h(x)$ (b) Dual space in slab I_j

Fig. 3. Optimal placement on an edge uv. (a) Corner tent functions $f_c(x)$, crossover breakpoints b_c, slabs I_j, upper envelope $h(x)$ (purple), and optimum $g^* = q(x^*)$. (b) In each slab I_j, point–line duality maps $y = mx + b$ to $(m, -b)$, whose lower convex hull gives the active envelope lines.

B. Multi-Corner Optimization as a Network 1-Center Problem

On edge uv, the position that minimizes the worst-case ratio solves

$$\min_{x \in [0,\ell]} \; h(x), \quad h(x) \;=\; \max_{c \in \mathcal{C}} \; f_c(x). \tag{14}$$

Over the whole network this is a *weighted continuous 1-center problem* [66], [67] with the entry-node sets Z_c as demand points, weighted at each frame by $w_c = 1/L_c$. Because each f_c has a single breakpoint b_c, the upper envelope h is minimized exactly in two stages [68].

a) Stage A: Slab decomposition.: The sorted breakpoints $\{b_c\}$ partition $[0, \ell]$ into at most $|\mathcal{C}| + 1$ slabs $I_1, I_2, \ldots$ (Fig. 3(a)). No tent changes slope inside a slab, so on each I_j the objective h is the upper envelope of $|\mathcal{C}|$ lines.

b) Stage B: Lowest point of a line envelope via duality.: Under point–line duality, a line $y = mx + b$ maps to the point $(m, -b)$, and the upper envelope of the lines corresponds to the lower convex hull of the dual points (Fig. 3(b)). The hull vertices are the active lines and the hull edges give the switch-over abscissae between them. Since the envelope is convex on a slab, its minimum lies at a switch-over or at a slab boundary, and scanning the hull vertices finds it. The hull costs $O(|\mathcal{C}| \log |\mathcal{C}|)$ per slab, and the slab count governs the total.

Proposition 1. *For any roadmap edge $uv \in E$, the minimum of $h(x) = \max_{c \in \mathcal{C}} f_c(x)$ over $[0, \ell]$ is computed exactly in $O\big(|\mathcal{C}|^2 \log |\mathcal{C}|\big)$ time by the slab–duality procedure.*

Proof. At most $|\mathcal{C}| + 1$ slabs, each with an $O(|\mathcal{C}| \log |\mathcal{C}|)$ hull and an $O(|\mathcal{C}|)$ scan of its vertices, give the bound, and the scan is exact because the slab minimum is attained at a scanned point. □

C. Global Roadmap Optimum

Evaluating the per-edge minimizer x_η^* on every edge $\eta \in E$ gives

$$\eta^* = \arg\min_{\eta \in E} \; h_\eta(x_\eta^*), \tag{15}$$

$$g^* = q_{\eta^*}(x_{\eta^*}^*), \tag{16}$$

where h_η is the envelope on edge η. Because $g^*(e)$ minimizes a min–max objective over a network, it can jump between distant edges when the bottleneck corner changes as the evader moves. The controller of Section VI handles both smooth drift and such jumps.

D. Spending Timing Slack on Line of Sight

The placement $g^*(e)$ addresses timing only, but it constrains only where the pursuer must be able to arrive, and several roadmap positions often have worst-case ratios close to $\alpha^*(e)$. For $\varepsilon \geq 0$, the *ε-slack set*

$$V_\varepsilon(e) \;=\; \big\{\, q \in V : \max_{c\in\mathcal{C}} \alpha_c(q,e) \leq (1+\varepsilon)\,\alpha^*(e) \,\big\} \qquad (17)$$

collects the roadmap vertices within a factor $(1+\varepsilon)$ of the optimum. The slack variant ***Min-Max +Vis*** uses this freedom for sight. At each frame, if $g^*(e)$ lies in $\mathrm{Vis}(e)$, the pursuer tracks $g^*(e)$. Otherwise it selects the vertex $q \in V_\varepsilon(e) \cap \mathrm{Vis}(e)$ with the lowest worst-case ratio (we use $\varepsilon = 0.1$), falling back to $g^*(e)$ if no slack vertex sees the evader. This costs one visibility polygon $\mathrm{Vis}(e)$ per frame. The same $(1+\varepsilon)$ relaxation is used in the team program of Section VII, so single- and multi-agent tracking follow one principle: secure the worst-case timing race first, then spend the slack on line of sight.

E. Computational Complexity and Runtime

Each frame computes the geodesic distances L_c on the n-vertex visibility graph in $O(n^2)$ and then minimizes the envelope on all $|E|$ edges in $O(|E|\cdot|\mathcal{C}|^2 \log|\mathcal{C}|)$ by Proposition 1, with the distances $d_{\mathcal{G}}(u, \mathcal{A}(c))$ read from the table.

In our single-threaded Python implementation (median over 100 evader configurations), a frame takes 37 ms on the smallest benchmark (`poly3`: $|E| = 94$, $|\mathcal{C}| = 10$), 147 ms on `poly9` ($|E| = 262$, $|\mathcal{C}| = 17$), and 580 ms on the largest (`poly8`: $|E| = 698$, $|\mathcal{C}| = 29$). The envelope pass dominates, at 32, 126, and 482 ms against 6, 21, and 53 ms for the geodesic queries, because the interpreted code rebuilds $|E|\,|\mathcal{C}|$ tents at every step. This cost sets the 5-Hz planning rate of the closed-loop deployment in Section VIII-H.

VI. Roadmap-Constrained Pursuer Motion

Given the target placement $g^*(e(t))$ (or its slack refinement) at frame t, the pursuer must move from $p(t) \in \mathcal{G}$ toward it along the edges of $\mathcal{G}$. The controller must follow genuine jumps of g^* between distant edges while ignoring the artificial goal motion that replanning itself introduces.

a) Oscillation under naive replanning.: A controller that calls $\pi = \mathrm{Dijkstra}(\mathcal{G}, p, g^*)$ from scratch at every frame and steps along the result chatters. Both $p(t)$ and g^* usually lie in the interior of edges, so inserting them as virtual vertices splits their host edges, and a pursuer moving along an edge sees the vertex it just passed reappear as the first waypoint and reverses direction at every tick.

b) Stable-node roadmap controller.: We split the shortest path into a committed *base path* π_{base} of permanent roadmap vertices and the terminal goal g^*, and keep π_{base} across frames under three rules. (i) *Acceptance pruning* removes the leading vertex once the pursuer is within 0.5 world units of it. (ii) *Backtracking elimination* strips the most recently visited vertex when it reappears at the head of a candidate path only because of edge subdivision. (iii) *In-place tail updating* replaces the suffix of π_{base} beyond the current waypoint when the goal drifts without changing the heading. When pruning empties π_{base}, pursuer and goal share an edge and the pursuer moves directly toward g^*. Only a change of the required leading vertex, that is, a genuine jump of g^*, triggers a reroute.

c) Roadmap invariance.: Each update advances the pursuer along π_{base} up to its speed budget s_p, taking a fractional step on the terminal segment only when g^* is collinear with the current edge, and halting at a non-collinear vertex until the next frame. Thus $p(t)$ always lies on $\mathcal{G}$ and never cuts through the polygon interior. The halts slightly lower the realized speed below s_p, so the trajectory metrics of Section VIII are conservative relative to the oracle bound $s^\star$.

d) Per-frame complexity.: Each update executes one Dijkstra query on $\mathcal{G}$ and $O(|V|)$ path filtering, for $O((|V|+|E|)\log|V|)$ time per frame.

VII. Extension to Multiple Pursuers and Evaders

A single pursuer needs $s^\star \geq 1.42$ on every benchmark (Section VIII-E) because one guard position must contest the race to every corner at once. In this section, we extend the framework to k pursuers by partitioning the corners $\mathcal{C}$ into k disjoint groups, which decouples the team into k independent single-pursuer problems with the guard set, roadmap, optimizer, and controller unchanged. An exact mixed-integer linear program (MILP) chooses the partition for speed first and persistent line of sight second.

A. Team Formulation and Sight Objective

a) Corner partition and team ratio.: Let $\mathcal{C} = \mathcal{C}_1 \,\dot\cup\, \cdots \,\dot\cup\, \mathcal{C}_k$ be a partition in which pursuer i guards the corners in $\mathcal{C}_i$. With pursuers at $p_1, \ldots, p_k \in \mathcal{G}$, an escape to corner c is cut off if any pursuer reaches $\mathcal{A}(c)$ first, so the *team ratio* at c is $\min_i \alpha_c(p_i, e)$. Pursuer i's own guard position and value are

$$g_i^*(e) \;=\; \arg\min_{q\in\mathcal{G}} \max_{c\in\mathcal{C}_i} \alpha_c(q,e), \qquad (18)$$

with $\alpha^*_{\mathcal{C}_i}(e) = \max_{c\in\mathcal{C}_i} \alpha_c(g_i^*(e), e)$, and the *team critical speed ratio*, the k-pursuer analogue of (8), is

$$s^\star_k \;=\; \sup_{e\in\mathcal{P}} \max_{i\in\{1,\ldots,k\}} \alpha^*_{\mathcal{C}_i}(e). \qquad (19)$$

b) Persistently visible roadmap.: Let $e_c(t)$ be the evader's position along its geodesic from e to c. The *persistently visible roadmap* of this escape is the set of roadmap positions from which a stationary guard sees the evader throughout its run to c:

$$\mathcal{V}(e,c) \;=\; \big\{\, q \in \mathcal{G} : \overline{q\, e_c(t)} \subset \mathcal{P} \;\; \forall t \,\big\}. \qquad (20)$$

c) Discretization and team sight score.: To obtain a finite program, we restrict pursuer positions to a sample set $Q \subset \mathcal{G}$ (the roadmap vertices plus edge samples every 15 world units) and evader positions to a uniform grid $\Omega \subset \mathcal{P}$. Let $q_i(e) \in Q$ be the position pursuer i occupies when the evader is at $e \in \Omega$. Closer corners pose the more immediate threat, so we weight each corner inversely by its distance from the evader,

$$w_c(e) = \frac{1/L_c}{\sum_{c'\in\mathcal{C}} 1/L_{c'}}. \qquad (21)$$

The *team sight score* of a partition is the weighted fraction of owned escapes that are persistently visible from the owner's position. Averaged over the grid, it is

$$\Lambda \;=\; \frac{1}{|\Omega|}\sum_{e\in\Omega}\sum_{i=1}^{k}\sum_{c\in\mathcal{C}_i} w_c(e)\,\mathbb{1}\big[\,q_i(e)\in\mathcal{V}(e,c)\,\big]. \tag{22}$$

For a speed threshold $\theta>0$, $B_\theta(e,q)=\{c\in\mathcal{C}:\alpha_c(q,e)>\theta\}$ is the set of corners a guard at q cannot reach in time when the evader is at e.

B. Separability

Choosing all k positions jointly at every frame is a continuous network k-center problem, NP-hard in general [67]. A fixed partition removes the coupling: restricting pursuer i to $\mathcal{C}_i$ only drops tents from the envelope of Section V, so each $g_i^*(e)$ is computed by the same slab–duality algorithm, and choosing the partition offline is a static task allocation [69].

Proposition 2 (Separability and monotonicity)**.** *Under any fixed partition, the k problems* (18) *are independent, cost no more per frame in total than the single-pursuer problem, and never worsen the worst-case ratio relative to a single pursuer:*

$$\max_{c\in\mathcal{C}}\min_{i}\alpha_c(g_i^*(e),e)\;\le\;\max_{i}\alpha^*_{\mathcal{C}_i}(e)\;\le\;\alpha^*_{\mathcal{C}}(e).$$

Proof. The cost bound follows from Proposition 1 and $\sum_i|\mathcal{C}_i|^2\le|\mathcal{C}|^2$. The first inequality holds because every corner belongs to some $\mathcal{C}_i$, the second because the single-pursuer optimum $g^*(e)$ is feasible for every subproblem. □

Online, each pursuer tracks its own $g_i^*(e(t))$ with the stable-node controller and no coordination beyond the shared evader state. What remains is to choose the partition, judged by $s_k^\star$ and Λ, which can conflict because the position that arrives in time may not see.

C. Computing the Persistently Visible Roadmap

Evaluating Λ requires $\mathcal{V}(e,c)$ for every grid position and corner. The following lemma reduces this trajectory visibility test to finitely many visibility polygons and cone exclusions at the bends of e_c.

Lemma 1. *Let $\mathcal{P}$ be simply connected and let the geodesic e_c from e to c have vertices $e=u_0,u_1,\dots,u_r=c$, each interior bend u_j ($j\ge1$) a reflex corner. A guard at $q\in\mathcal{P}$ sees the entire path if and only if $q\in\bigcap_{j=0}^{r}\mathrm{Vis}(u_j)$ and, at each u_j with $j\ge1$, the obstacle wedge at u_j does not occlude the initial segment of either incident leg from q. The latter occurs if and only if q lies in the open cone at u_j spanned by the reflected leg direction and the wedge edge opposite that leg.*

Proof. Suppose q sees u_j and u_{j+1}, so the sides $\overline{qu_j}$ and $\overline{qu_{j+1}}$ of the triangle qu_ju_{j+1} lie in $\mathcal{P}$. Since $\mathcal{P}$ is simply connected, its exterior can intrude into the triangle only through a reflex vertex on its boundary, namely the wedge at u_j or u_{j+1} (q and u_0 are interior points). The wedge at u_j shares its apex with the leg $\overline{u_ju_{j+1}}$. By the ray-extension argument of Lemma **??**, it occludes a contiguous initial stretch of the leg if and only if the leg direction lies in the shadow cone of u_j with respect to q. Rewriting that condition in terms of q gives the stated cone. If neither wedge intrudes, the triangle lies in $\mathcal{P}$ and q sees the whole leg. □

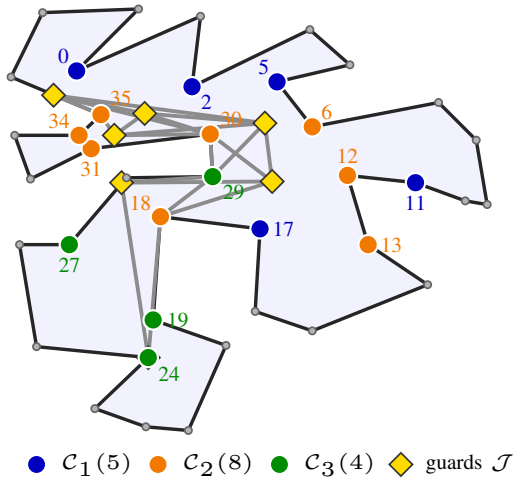


Fig. 4. Optimal corner partition for $k=3$ pursuers on `poly9` ($n=39$, $|\mathcal{C}|=17$) from (23)–(25). Gray segments show the shared patrol roadmap $\mathcal{G}$, and diamonds denote the guard set $\mathcal{J}$. Because the central roadmap hub reaches multiple chambers quickly, the optimal groups are not spatially contiguous.

The implementation enforces the cone condition by also requiring visibility of two witness points a small step along the incident legs at each bend. All viewpoints form a small static set per polygon, so caching $\mathcal{G}\cap\mathrm{Vis}(v)$ for each makes every $\mathcal{V}(e,c)$ a fast set intersection. On `poly9`, the cone condition tightens $\mathcal{V}(e,c)$ by 3.3% on average relative to intersecting the vertex visibility polygons alone.

D. Optimal Corner Partitioning via Mixed-Integer Programming

Over $Q\times\Omega$, the partition problem uses three sets of binary variables. x_{ci} indicates that corner c is assigned to pursuer i, z_{eiq} that pursuer i stands at $q\in Q$ when the evader is at $e\in\Omega$, and v_{eic} that pursuer i owns c and sees the whole escape e_c from its position. For a speed bound $\theta>0$, the MILP is

$$\max\ \tfrac{1}{|\Omega|}\textstyle\sum_{e,i,c} w_c(e)\,v_{eic} \tag{23}$$

$$\text{s.t. }\textstyle\sum_{i=1}^{k}x_{ci}=1\ \ \forall c,\qquad \sum_{q\in Q}z_{eiq}=1\ \ \forall e,i,$$
$$\textstyle\sum_{c\in B_\theta(e,q)}x_{ci}+|B_\theta(e,q)|\,z_{eiq}\le|B_\theta(e,q)|\ \ \forall e,i,q, \tag{24}$$

$$v_{eic}\le x_{ci},\quad v_{eic}\le\textstyle\sum_{q:\,q\in\mathcal{V}(e,c)}z_{eiq}\ \ \forall e,i,c. \tag{25}$$

Constraint (24) forbids a pursuer standing at q from owning any corner in $B_\theta(e,q)$, so a feasible solution certifies $s_k^\star\le\theta$ on the discretization. Constraint (25) grants sight credit for c only to its owner, standing in $\mathcal{V}(e,c)$.

We find the minimum feasible $s_k^\star$ over the discretization by bisection on θ. Solving the full program at $\theta=s_k^\star$ gives the least-speed, most-sight partition, and solving at $\theta=(1+\varepsilon)s_k^\star$ traces the Pareto front between speed margin and persistent visibility. Every returned partition is re-scored with the continuous optimizer of Section V on a fine 25×25 grid. On `poly9`, the MILP has about 34,000 binary variables,

solves in under 10 s with HiGHS, and returns the three-pursuer partition of Fig. 4.

a) Co-visibility clustering heuristic.: For larger instances or initialization without a solver, we also define a heuristic based on the co-visibility affinity $\rho(c, c') = |\Omega|^{-1} \sum_{e \in \Omega} |\mathcal{V}(e, c) \cap \mathcal{V}(e, c')|$, the expected length of roadmap that sees escapes to both c and c'. Average-linkage clustering on the dissimilarity $1 - \rho / \max \rho$, followed by a greedy swap search on $s_k^\star$, gives a partition in seconds.

E. Extension to Multiple Evaders

The evader enters the optimization only through the distances L_c, so m evaders $e_1, \ldots, e_m$ require no change to roadmap, partition, or controller. The responsible pursuer must beat whichever evader can reach c first, so we set

$$L_c = \min_{j \in \{1, \ldots, m\}} d_{\text{geo}}(e_j, c) \tag{26}$$

and run the envelope optimization and controller unchanged. Pursuers are assigned to corners rather than to evaders, so no matching is needed and $m > k$ is allowed. Several corners may then be threatened at once, so the realized worst-case ratio grows with m, and we measure multi-evader performance by the fraction of evaders in view of at least one pursuer.

VIII. Simulation and Experiments

In this section, we evaluate the guard-set construction, per-frame placement, ε-slack layer, multi-pursuer partitioning, and closed-loop flight control on nine polygonal environments.

A. Implementation and Setup

The simulation is implemented in Python with Shapely [70] for geometric operations, VisiLibity [71] for exact visibility polygons, and SciPy [64] for Floyd–Warshall and the HiGHS-based integer programs. Unless stated otherwise, the pursuer and evader speeds are $s_p = 0.8$ and $s_e = 1.0$ world units per frame, the regime $s_p \leq s_e$ in which the pursuer has no speed advantage.

B. Baselines and Evader Models

All roadmap-based strategies share the guard set $\mathcal{J}$, the roadmap $\mathcal{G}$, and the stable-node controller, and differ only in their per-frame target:

- **Min-Max α-Guard (proposed)**: the timing-optimal placement $g^*(e)$ of (7).
- **Min-Max +Vis (proposed)**: the ε-slack placement of Section V-D with $\varepsilon = 0.1$.
- **Geo-Follow**: the roadmap vertex geodesically closest to the evader.
- **Greedy-LOS**: the closest roadmap vertex that currently sees the evader (Geo-Follow when none does). This is a sight-only ablation with no timing term.
- **TSP-Patrol** [51]: an exact minimum-length (Held–Karp) tour over $\mathcal{J}$, independent of the evader.
- **Kernel-Weighted Control** [8]: a free (off-roadmap) pursuer anchored at $\mathcal{J}$ whose velocity is a Gaussian-weighted sum of unit directions toward the active gap-corners.

We use three evader models. (1) The **skeleton evader** traverses an interior Voronoi skeleton of $\mathcal{P}$, choosing distant waypoints it has not recently visited, and ignores the pursuer. (2) The **adversarial evader** reads the pursuer's live position at every frame, heads for the corner $c^* = \arg\max_c \alpha_c(p, e)$ with the worst ratio, and holds a 0.5-unit standoff from it. (3) The **escaping evader** executes the escape of Lemma **??**(ii). Whenever it detects a winning race ($\alpha_c(p, e) > s_p / s_e$ for some c), it commits to that corner, runs its geodesic to c, and steps into the shadow of c relative to the pursuer's live position. With no winning race it behaves as the adversarial evader.

Each configuration is run for 50 paired random seeds of $T = 600$ frames. A seed fixes the evader's initial position and waypoint sequence across all strategies, so comparisons are paired. We report the time-averaged worst-corner ratio $\bar{\alpha} = \frac{1}{T} \sum_{t=1}^{T} \max_c \alpha_c(p(t), e(t))$, the line-of-sight coverage %LOS (fraction of frames with $e(t) \in \text{Vis}(p(t))$), and the escape count N_{esc} (contiguous occlusion intervals longer than $\tau = 30$ frames, or 1 s). Significance is assessed by paired Wilcoxon signed-rank tests with Holm correction across environments, and "significant" below always refers to this test.

C. Benchmark Environments



(a) `poly3` (23, 10) (b) `poly8` (60, 29) (c) `poly9` (39, 17)

Fig. 5. Three of the nine benchmark polygons ($n, |\mathcal{C}|$ shown); Table II lists all nine. Red markers denote reflex corners $\mathcal{C}$ and gray markers convex vertices.

We evaluate on nine non-convex polygons with $23 \leq n \leq 60$ vertices and $10 \leq |\mathcal{C}| \leq 29$ reflex corners (Fig. 5, Table II). On every environment, the exact set cover reduces thousands of KER candidates ($360 \leq |\mathcal{K}| \leq 2{,}784$) to guard sets of $5 \leq |\mathcal{J}| \leq 13$ patrol nodes. Their visibility polygons cover 95.6%–100% of the polygon area, and the shortfall is numerical (Section VIII-I).

D. Single-Pursuer Tracking Performance

a) Skeleton evader.: Table I reports the single-pursuer results. Both proposed methods achieve the lowest $\bar{\alpha}$ on all nine environments (narrowly ahead of Geo-Follow on `poly6` and `poly8`). They reduce $\bar{\alpha}$ by up to 42% relative to Geo-Follow (`poly2`) and beat the off-roadmap Kernel-Weighted controller on eight of nine polygons, with a tie on `poly6`. The reduction is significant on six of nine environments against both Geo-Follow and Greedy-LOS.

TABLE I

SINGLE-PURSUER TRACKING AGAINST THE SKELETON EVADER: MEAN ± STD OVER 50 SHARED SEEDS ($T = 600$, $s_p/s_e = 0.8$). BEST PER ROW IN BOLD. ESCAPES STAYED BELOW 1.6 PER TRIAL FOR EVERY REACTIVE STRATEGY (TSP-PATROL: 0.9–1.7).

	$\bar{\alpha}$ (lower is better)						%LOS (higher is better)					
	Min-Max	+Vis	Geo-Follow	Greedy-LOS	TSP	Kernel	Min-Max	+Vis	Geo-Follow	Greedy-LOS	TSP	Kernel
poly1	2.49 ± 1.60	$\mathbf{2.48 \pm 1.57}$	3.19 ± 1.49	3.15 ± 1.51	7.31 ± 3.10	4.36 ± 2.35	85.8 ± 19.6	$\mathbf{87.9 \pm 19.1}$	86.5 ± 16.2	87.8 ± 16.4	42.2 ± 28.5	57.3 ± 23.3
poly2	$\mathbf{1.39 \pm 0.36}$	1.39 ± 0.36	2.41 ± 1.18	2.33 ± 1.25	5.11 ± 2.83	3.62 ± 1.74	95.6 ± 14.1	$\mathbf{98.8 \pm 3.6}$	85.8 ± 23.6	92.0 ± 13.7	64.5 ± 31.9	82.0 ± 24.1
poly3	1.56 ± 0.66	$\mathbf{1.56 \pm 0.64}$	1.69 ± 0.74	1.60 ± 0.67	4.90 ± 2.16	2.47 ± 1.25	88.8 ± 21.2	90.8 ± 17.6	93.0 ± 20.4	$\mathbf{98.7 \pm 6.2}$	30.8 ± 20.1	77.2 ± 24.9
poly4	$\mathbf{1.54 \pm 0.42}$	1.59 ± 0.45	2.47 ± 0.95	2.47 ± 0.93	5.71 ± 2.70	2.75 ± 1.05	88.4 ± 18.9	90.0 ± 18.7	97.0 ± 9.8	$\mathbf{97.7 \pm 9.0}$	48.8 ± 21.9	91.7 ± 22.2
poly5	2.25 ± 0.85	$\mathbf{2.09 \pm 0.78}$	2.93 ± 1.36	2.89 ± 1.32	4.51 ± 1.44	3.04 ± 0.98	86.0 ± 24.3	$\mathbf{93.7 \pm 12.5}$	92.5 ± 12.1	93.0 ± 11.4	56.4 ± 24.1	90.6 ± 22.3
poly6	3.10 ± 1.29	$\mathbf{3.07 \pm 1.26}$	3.11 ± 1.28	3.12 ± 1.28	4.66 ± 1.42	3.07 ± 1.04	82.4 ± 22.0	84.3 ± 19.3	$\mathbf{87.5 \pm 17.7}$	87.2 ± 18.0	57.8 ± 22.3	83.9 ± 22.3
poly7	$\mathbf{1.73 \pm 0.78}$	1.73 ± 0.76	2.16 ± 1.04	2.19 ± 1.03	5.83 ± 3.12	2.57 ± 1.14	83.3 ± 26.8	86.2 ± 23.4	89.2 ± 23.9	92.0 ± 19.3	62.1 ± 35.8	$\mathbf{93.7 \pm 11.7}$
poly8	$\mathbf{4.58 \pm 2.50}$	4.74 ± 2.53	4.81 ± 2.65	4.66 ± 2.61	10.54 ± 4.49	6.49 ± 3.92	64.8 ± 20.0	$\mathbf{65.6 \pm 20.6}$	61.9 ± 22.7	63.1 ± 23.1	10.9 ± 14.1	45.9 ± 23.0
poly9	$\mathbf{1.51 \pm 0.65}$	1.52 ± 0.67	1.93 ± 0.92	1.95 ± 0.93	5.14 ± 2.25	2.37 ± 1.24	86.8 ± 16.3	87.5 ± 14.5	88.7 ± 21.0	$\mathbf{94.4 \pm 9.9}$	25.2 ± 30.5	81.3 ± 21.5

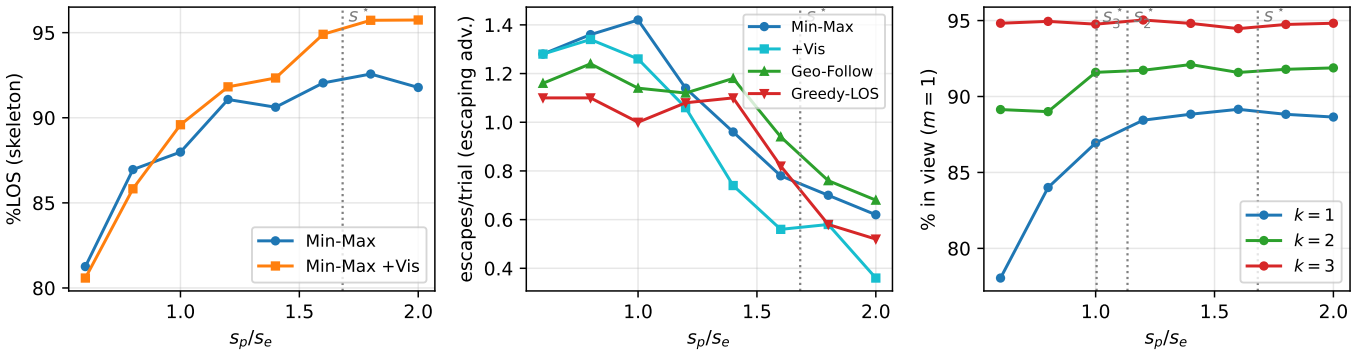


Fig. 6. Speed-ratio sweep on `poly9` (50 seeds per ratio): %LOS against the skeleton evader (left), escapes per trial against the *escaping* adversary (middle), and fraction of one evader in view for k-pursuer teams on the MILP partitions (right). Dotted lines mark $s^\star$ and $s^\star_k$. The middle panel shows the predicted crossover: below the evader's speed, shadowing concedes slightly fewer escapes, and with speed margin ($s_p/s_e \geq 1.4$) the placement strategies deny significantly more.

TABLE II

BENCHMARK ENVIRONMENTS. n, $|\mathcal{C}|$ AFTER BOUNDARY CLEANING; $|\mathcal{K}|$ BEFORE DOMINATION PRUNING; $|\mathcal{J}|$ SELECTED GUARDS; $|E|$ PATROL GRAPH EDGES; %AREA SEEN BY $\mathcal{J}$; $s^\star$ THE CRITICAL SPEED RATIO OF SECTION III-B (25×25 EVADER GRID).

	n	$\lvert\mathcal{C}\rvert$	$\lvert\mathcal{K}\rvert$	$\lvert\mathcal{J}\rvert$	$\lvert E\rvert$	%Area	$s^\star$
poly1	48	22	1534	9	540	100.0	1.907
poly2	47	23	2249	9	554	99.4	1.677
poly3	23	10	360	5	94	100.0	1.833
poly4	47	20	2784	5	204	98.9	2.713
poly5	38	17	1135	8	416	100.0	1.945
poly6	39	17	996	7	364	97.2	2.168
poly7	30	12	526	7	284	97.6	1.418
poly8	60	29	1716	13	698	95.6	1.738
poly9	39	17	1477	7	262	100.0	1.682

TABLE III

ADVERSARIAL EVADER: %LOS AND MEAN ESCAPES N_{ESC} PER TRIAL OVER 50 SEEDS. $\bar{\alpha}$ IS OMITTED BECAUSE THE EVADER HOLDS A 0.5-UNIT STANDOFF AT A CORNER ($L_c \to 0.5$).

	%LOS					N_{esc}				
	MM	+V	GF	GL	TSP	MM	+V	GF	GL	TSP
poly1	86.2	85.0	93.4	93.6	25.3	0.64	0.64	0.30	0.30	1.04
poly2	86.1	86.6	87.0	93.7	68.8	0.54	0.48	0.34	0.26	0.80
poly3	96.7	96.9	90.5	98.5	36.6	0.12	0.12	0.16	0.08	0.98
poly4	91.3	93.9	99.8	99.8	40.4	0.24	0.24	0.02	0.02	1.14
poly5	89.1	90.1	95.8	96.2	38.3	0.34	0.30	0.10	0.08	0.96
poly6	92.2	92.6	96.7	98.0	38.8	0.22	0.18	0.14	0.06	1.24
poly7	84.0	89.5	95.4	95.6	37.4	0.46	0.46	0.20	0.18	0.74
poly8	93.7	93.8	94.4	94.4	21.5	0.30	0.30	0.30	0.30	1.80
poly9	82.1	80.8	77.1	87.6	23.5	0.56	0.56	0.46	0.36	0.84

The ε-slack layer improves sight at no timing cost: Min-Max +Vis raises %LOS over plain Min-Max on every environment (by 0.7 to 7.7 points) while changing $\bar{\alpha}$ by at most 0.16. Against Geo-Follow, its %LOS is indistinguishable on seven of nine environments, with one significant difference in each direction (+13.0 points for +Vis on `poly2`, −7.0 on `poly4`). Against this non-adversarial evader, the sight-only Greedy-LOS reaches high %LOS (ahead of +Vis on three environments by up to 7.9 points, behind on one) but pays a significant timing penalty in $\bar{\alpha}$ on six of nine. That penalty matters against an escaping adversary, as the speed sweep below shows. TSP-Patrol is worst on both metrics everywhere.

b) Adversarial evader.: Table III reports results against the standoff adversary. Since $s_p/s_e = 0.8$ lies below $s^\star$ on every polygon, no single roadmap position can cover all corners, and the adversary always finds a race it wins. TSP-Patrol, which ignores the evader, suffers 0.74–1.80 escapes per trial with only 21.5%–68.8% %LOS, whereas the reactive strategies hold 77.1%–99.8% %LOS with at most 0.64 escapes. The followers show slightly fewer escapes than the proposed variants at this ratio, because the standoff evader parks near a corner without crossing into its shadow and shadowing it closely is enough. The speed sweep below shows where this trade fails.

E. Critical Speed Bounds and Speed Sweep

We evaluate $s^\star$ of (8) on a 25×25 grid (208–333 interior samples per polygon). The last column of Table II lists it. It ranges from 1.418 (`poly7`) to 2.713 (`poly4`), so on no benchmark does a pursuer at or below the evader's speed satisfy

the necessary cutoff condition everywhere. The bound is set by visibility bottlenecks rather than size: one deep recess makes `poly4` the most demanding environment although it needs only five guards. Our nominal trials at $s_p/s_e = 0.8 < s^\star$ therefore operate in a regime where the proposed strategies minimize worst-case exposure rather than eliminating it.

To see how performance changes with speed margin, we sweep s_p/s_e from 0.6 to 2.0 on `poly9` ($s^\star = 1.682$, 50 seeds per ratio, Fig. 6). Against the skeleton evader (left), Min-Max +Vis raises %LOS from 81% to 96% and cuts escapes threefold. Its margin over plain Min-Max widens with speed, since a faster pursuer has more vertices inside the ε-slack band.

Against the escaping adversary (middle), the two-sided crossover predicted by the formulation appears. For $s_p/s_e \leq 1.0$, the pursuer cannot win the races, so leaving the evader's vicinity to guard distant corners is counterproductive: pooled over these ratios, Greedy-LOS concedes 1.07 escapes per trial against 1.29 for Min-Max +Vis (paired $p = 0.007$). For $s_p/s_e \geq 1.4$ the order reverses. Min-Max +Vis has the lowest escape rate of all methods at 0.56 per trial, significantly below Greedy-LOS (0.76, $p = 0.013$), Geo-Follow (0.89, $p = 10^{-4}$), and plain Min-Max (0.77, $p < 10^{-4}$). It also leads in %LOS. With speed margin, timing placement wins the corner races and the slack layer turns the won races into sight.

F. Multi-Pursuer Team Tracking

We solve the MILP (23)–(25) on `poly9` with an 8×8 evader grid and $|Q| = 331$ roadmap samples, and re-score every partition on the 25×25 grid of Table II. Table IV compares the MILP with the clustering heuristic of Section VII-D for $k \in \{1, 2, 3\}$.

TABLE IV
CORNER PARTITIONS ON POLY9: TEAM SPEED RATIO $s_k^\star$ (19) AND SIGHT SCORE Λ (22).

k	Partition Method	$s_k^\star$	Λ
1	Single pursuer	1.682	0.712
2	Heuristic (raw clustering)	1.682	—
2	Heuristic (swap-refined)	1.149	0.755
2	**MILP (min speed)**	**1.134**	0.755
2	MILP (bound $+10\%$)	1.170	**0.792**
3	Heuristic (raw clustering)	1.682	—
3	Heuristic (swap-refined)	1.008	0.757
3	**MILP (min speed)**	**1.004**	0.756
3	MILP (bound $+10\%$)	1.033	**0.834**

Partitioning the corners reduces the necessary speed ratio from 1.682 for one pursuer to 1.134 for $k = 2$ and 1.004 for $k = 3$: on the validation grid, three equal-speed pursuers remove the single-agent timing obstruction. Raw co-visibility clustering leaves $s_k^\star$ at 1.682, because it splits off one or two peripheral corners and leaves the rest in one group. Corners that are *seen* from common roadmap segments are not the corners that must be *reached* together. The swap search brings the heuristic within 1.5% of the MILP bound. The advantage of the MILP is the explicit speed–sight trade. Relaxing the speed bound by 10% raises Λ from 0.755 to 0.792 ($k = 2$) and from 0.756 to 0.834 ($k = 3$) at a cost of at most 0.036 in $s_k^\star$. As Fig. 4 shows, the optimal groups are not spatially contiguous: the roadmap's central hub reaches corners in different chambers quickly, so the MILP spreads each group across chambers to balance worst-case travel times.

G. Multiple Pursuers Against Multiple Evaders

Table V evaluates $k \in \{1, 2, 3\}$ pursuers against $m \in \{1, 2, 3\}$ skeleton evaders on `poly9` with the nearest-evader rule (26). The oracle columns give the 95th percentile and mean of the team ratio over 300 sampled evader configurations with a 2.1-unit corner standoff, the closest approach of the 25×25 grid. The trial columns average 50 seeds of 600 frames at $s_p = 0.8$, $s_e = 1.0$.

TABLE V
k PURSUERS AGAINST m EVADERS ON POLY9 (MILP PARTITIONS). ORACLE: TEAM α OVER 300 SAMPLED CONFIGURATIONS. TRIALS: MEAN $\pm$ STD OVER 50 SEEDS.

		Oracle α		Trials		
k	m	p95	mean	mean α	% in view	N_{esc}
1	1	1.27	0.92	1.45 ± 0.57	82.4 ± 19.1	1.0 ± 1.1
1	2	4.54	2.07	2.87 ± 1.23	56.5 ± 28.5	2.7 ± 1.5
1	3	6.18	3.10	3.31 ± 1.21	48.2 ± 18.9	5.0 ± 2.1
2	1	1.05	0.77	1.24 ± 0.57	88.3 ± 14.4	0.8 ± 0.9
2	2	2.83	1.39	2.02 ± 0.79	81.4 ± 17.2	1.7 ± 1.4
2	3	3.53	1.92	2.62 ± 0.71	79.6 ± 16.9	2.8 ± 2.3
3	1	0.92	0.67	0.85 ± 0.35	95.0 ± 10.8	0.3 ± 0.7
3	2	2.39	1.17	1.35 ± 0.48	90.6 ± 12.9	1.0 ± 1.2
3	3	3.11	1.58	1.86 ± 0.50	89.1 ± 11.8	1.7 ± 1.7

Each additional evader costs about one pursuer. A single pursuer keeps 82.4% of one evader in view but only 48.2% of three, with 5.0 escapes per trial, whereas along the diagonal $k = m$ two pursuers keep 81.4% of two evaders in view and three keep 89.1% of three, with 1.7 escapes. If two evaders approach two corners of the same group $\mathcal{C}_i$, its owner cannot occupy both association regions, which raises the tail of the oracle ratio. Online corner reallocation is the natural remedy.

H. Case Study: Closed-Loop Hexacopter Deployment in Gazebo

To test the pipeline under vehicle dynamics and control delay, we deployed it unmodified as a 5-Hz ROS 2 node, with the envelope optimizer, ε-slack layer, and stable-node controller. The arena is a Gazebo model of `poly9` at 0.05 m per world unit, 45×40 m with 3 m walls. A differential-drive ground robot executing the skeleton evader policy drives on the floor while a hexacopter with per-rotor thrust dynamics and a $2.5\,\text{m/s}^2$ horizontal acceleration limit tracks the planner's roadmap position overhead through velocity-damped commands. Line of sight is evaluated in the horizontal plane at the evader's height. The loop closes both ways: the planner reads the UAV's odometry and holds its target whenever the hexacopter lags more than 2.0 m behind the planned state.

Over five 180-s flights per strategy (Fig. 7), both proposed methods fly within the airframe's limits. The acceleration

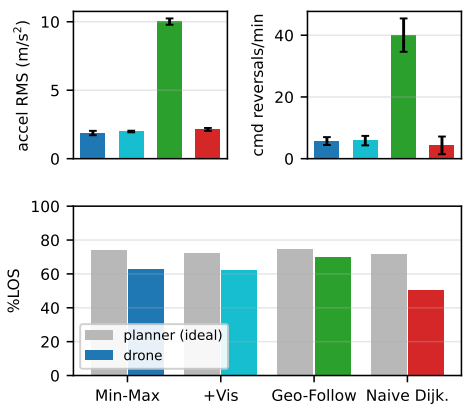


Fig. 7. Closed-loop Gazebo deployment on `poly9`, five 180-s flights per strategy (mean ± std): UAV acceleration RMS, commanded direction reversals per minute, and planar %LOS at the planner's state vs. the UAV's position.

RMS is 1.9–2.0 m/s^2 against the 2.5 m/s^2 limit, with about six commanded direction reversals per minute. The mean tracking lag is 1.2–1.3 m, the hold coupling triggers on 7%–8% of ticks, and the physical %LOS of 62%–63% is within ten points of the kinematic planner.

The baselines degrade physically. Geo-Follow jumps between roadmap vertices as the evader moves, which produces 40 direction reversals per minute and outruns the vehicle on 55% of ticks. The reversals excite attitude oscillations that raise the realized acceleration RMS to 10.0 m/s^2, four times the command-shaping limit. Replacing the stable-node controller by naive per-tick Dijkstra replanning (Section VI) causes continuous mid-edge direction flips and lowers physical %LOS to 50%, twelve points below Min-Max under identical dynamics.

I. Discussion

Several observations follow from the results, and they point to the limitations and extensions we discuss at the end.

1) Observations:

a) Transit lag versus the oracle bound.: The bound $s^\star$ credits the pursuer with standing at $g^*(e)$, whereas a real pursuer must travel along $\mathcal{G}$ as $g^*(e(t))$ moves. The peak ratio a moving pursuer faces in a trial relates to $s^\star$ in a way that varies by an order of magnitude across environments. On `poly4` (5 guards, 204 edges) the mean trial peak is 2.95 against $s^\star = 2.71$, on `poly9` it is 3.28 against 1.68, and on `poly8` (13 guards, 698 edges) it is 17.8 against 1.74. Even the trial average $\bar{\alpha}$ exceeds $s^\star$ on five of nine environments. The gap scales with the extent of the roadmap, not the size of the polygon, which motivates a transit-aware bound that accounts for how far $g^*(e)$ moves as the evader moves.

b) Operating regime of timing-optimal placement.: Two features of the speed sweep deserve comment. Below the evader's speed, leaving its neighborhood to defend distant corners opens a gap that an escaping adversary exploits, so in that regime, and against passive targets that never seek corners, sight-only shadowing is the better and cheaper policy. And the empirical crossover near $s_p/s_e \approx 1.2$ lies well below $s^\star = 1.682$, because $s^\star$ is a worst case over all evader positions while trial evaders start from random ones.

c) Residual occlusions above $s^\star$.: Even at $s_p/s_e = 2.0 > s^\star$ on `poly9`, the escaping evader obtains 0.36 escapes per trial against Min-Max +Vis while skeleton %LOS plateaus near 96%. This confirms the remark after (6): winning every arrival race prevents escapes at the corners but not occlusion-ray crossings away from them during transit.

d) Flatness of the envelope near the optimum.: Timing and sight can conflict in principle, but the results above show that a small slack in α buys considerable visibility, for the single pursuer and for the team alike. The reason is geometric: distant corners have large L_c and therefore shallow tent slopes $\pm 1/L_c$, so the envelope $h(x)$ has a wide, flat basin around $g^*(e)$ and $V_\varepsilon(e)$ contains many vertices from which to choose a clear line of sight.

e) Behavior of α at a standoff.: Since $\alpha_c = d_{\mathcal{G}}(p, \mathcal{A}(c))/L_c$ diverges as $L_c \to 0$, an evader parked next to a corner inflates α_c without entering the shadow, which is why Table III reports %LOS and N_{esc} only. The ratio makes $s^\star$ scale-invariant. Applications with stationary standoff targets can instead optimize the time margin $d_{\mathcal{G}}(p, \mathcal{A}(c))/s_p - L_c/s_e$, which stays bounded as $L_c \to 0$ and keeps the piecewise-linear tent structure on every edge.

2) Limitations and Extensions:

a) Incremental envelope maintenance.: The per-frame envelope rebuild of Section V-E sets the 5-Hz planning rate of the Gazebo experiments. Within a cell of the evader's shortest-path map only the weights $1/L_c$ change, and smoothly, so maintaining the envelope incrementally (or compiling the routine) would raise the planning rate to the 30–50-Hz control rate.

b) Grazing rays and grid certification.: KER points lie on extended reflex chords, so selected guards sit near visibility boundaries where sight lines graze reflex vertices. Floating-point visibility clipping then occasionally drops thin wedges in narrow corridors, up to 4.4% of the area on `poly8`, which a 0.5–1.5-unit polygon buffer recovers. Likewise, $s^\star$ and the team MILP are certified on grids, which gives lower bounds on the continuous quantities. Enumerating the arrangement vertices of the piecewise-analytic function $\alpha^*(e)$ would give exact certificates. Incorporating acceleration bounds into the edge travel times (piecewise-quadratic tents) and selecting the guard set for $s^\star$ rather than cardinality are further analytical extensions.

IX. Conclusion and Future Work

In this paper, we presented a framework for single- and multi-agent target tracking in polygons by pursuers confined to a patrol roadmap built from Keil-derived kernel candidates. From the occlusion geometry at each reflex corner, we showed that occupying the corner's association region before the evader reaches it is necessary to prevent escape there, and we scored this race by the ratio α. Guard selection from the candidate pool was solved as an exact 0/1 integer program, and the guards were connected by geodesic shortest paths. Per-frame placement was cast as a dynamically weighted network 1-center problem and solved exactly by an upper-envelope routine. Paired with a stable-node roadmap controller and an ε-slack visibility layer,

it attained the lowest or tied-lowest mean $\bar{\alpha}$ on nine benchmark polygons while matching the line-of-sight coverage of greedy followers.

The critical speed ratio $s^\star$ makes the limit of one pursuer explicit: a single roadmap-constrained guard needs 1.42–$2.71\times$ the evader's speed on our benchmarks to satisfy the necessary condition everywhere. A speed sweep against an escaping adversary confirmed the predicted crossover: with speed margin, the placement strategies deny significantly more escapes than reactive followers. Partitioning the corners among a team by an exact MILP reduced the required ratio on `poly9` from 1.682 to 1.004 with three pursuers, which kept 89.1% of three evaders in view. Closed-loop ROS 2 and Gazebo experiments showed that the pipeline flies within a hexacopter's acceleration limits, where a vertex-hopping follower does not.

In future work, we plan to incorporate transit lag and acceleration constraints into the tent functions to obtain transit-aware speed bounds. We also plan to select the guard set and roadmap to minimize $s^\star$ directly rather than guard count, to reallocate corners online against coordinated evader teams, and to validate the multi-pursuer framework on hardware.

REFERENCES

[1] S. M. LaValle, D. Lin, L. J. Guibas, J.-C. Latombe, and R. Motwani, "Finding an unpredictable target in a workspace with obstacles," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 1997, pp. 737–742.

[2] T. D. Parsons, "Pursuit-evasion in a graph," in *Theory and Applications of Graphs*, ser. Lecture Notes in Mathematics. Springer, 1978, vol. 642, pp. 426–441.

[3] I. Suzuki and M. Yamashita, "Searching for a mobile intruder in a polygonal region," *SIAM Journal on Computing*, vol. 21, no. 5, pp. 863–888, 1992.

[4] T. H. Chung, G. A. Hollinger, and V. Isler, "Search and pursuit-evasion in mobile robotics: A survey," *Autonomous Robots*, vol. 31, no. 4, pp. 299–316, 2011.

[5] R. Murrieta-Cid, R. Monroy, S. Hutchinson, and J.-P. Laumond, "A complexity result for the pursuit-evasion game of maintaining visibility of a moving evader," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2008, pp. 2657–2664.

[6] J. M. Keil, "Decomposing a polygon into simpler components," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 799–817, 1985.

[7] S. Mandal and S. Bhattacharya, "From distributed coverage to multi-agent target tracking," in *Proc. Int. Symp. Multi-Robot and Multi-Agent Systems (MRS)*, 2023, pp. 36–42.

[8] S. Mandal and S. Bhattacharya, "Scalable multi-agent surveillance: A kernel-based approach," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 8915–8921.

[9] C. Robin and S. Lacroix, "Multi-robot target detection and tracking: Taxonomy and survey," *Autonomous Robots*, vol. 40, no. 4, pp. 729–760, 2016.

[10] J. E. Littlewood, *A Mathematician's Miscellany*. London: Methuen, 1953.

[11] R. Isaacs, *Differential Games*. Wiley, 1965.

[12] D. Bhadauria, K. Klein, V. Isler, and S. Suri, "Capturing an evader in polygonal environments with obstacles: The full visibility case," *The International Journal of Robotics Research*, vol. 31, no. 10, pp. 1176–1189, 2012.

[13] N. Noori and V. Isler, "Lion and man with visibility in monotone polygons," *The International Journal of Robotics Research*, vol. 33, no. 1, pp. 155–181, 2014.

[14] L. J. Guibas, J.-C. Latombe, S. M. LaValle, D. Lin, and R. Motwani, "A visibility-based pursuit-evasion problem," *International Journal of Computational Geometry and Applications*, vol. 9, no. 4–5, pp. 471–493, 1999.

[15] B. P. Gerkey, S. Thrun, and G. Gordon, "Visibility-based pursuit-evasion with limited field of view," *The International Journal of Robotics Research*, vol. 25, no. 4, pp. 299–315, 2006.

[16] V. Isler, S. Kannan, and S. Khanna, "Randomized pursuit-evasion in a polygonal environment," *IEEE Transactions on Robotics*, vol. 21, no. 5, pp. 875–884, 2005.

[17] N. M. Stiffler and J. M. O'Kane, "Complete and optimal visibility-based pursuit-evasion," *The International Journal of Robotics Research*, vol. 36, no. 8, pp. 923–946, 2017.

[18] A. Kolling and S. Carpin, "Pursuit-evasion on trees by robot teams," *IEEE Transactions on Robotics*, vol. 26, no. 1, pp. 32–47, 2010.

[19] S. Bhattacharya and S. Hutchinson, "Approximation schemes for two-player pursuit evasion games with visibility constraints," in *Robotics: Science and Systems IV*, 2008.

[20] S. Bhattacharya, S. Candido, and S. Hutchinson, "Motion strategies for surveillance," in *Robotics: Science and Systems III*, 2007.

[21] S. Bhattacharya and S. Hutchinson, "On the existence of Nash equilibrium for a two-player pursuit-evasion game with visibility constraints," *The International Journal of Robotics Research*, vol. 29, no. 7, pp. 831–839, 2010.

[22] R. Zou and S. Bhattacharya, "On optimal pursuit trajectories for visibility-based target-tracking game," *IEEE Transactions on Robotics*, vol. 35, no. 2, pp. 449–465, 2019.

[23] S. Bhattacharya and S. Hutchinson, "A cell decomposition approach to visibility-based pursuit evasion among obstacles," *The International Journal of Robotics Research*, vol. 30, no. 14, pp. 1709–1727, 2011.

[24] S. Bhattacharya, T. Başar, and N. Hovakimyan, "A visibility-based pursuit-evasion game with a circular obstacle," *Journal of Optimization Theory and Applications*, vol. 171, no. 3, pp. 1071–1082, 2016.

[25] N. Karnad and V. Isler, "Lion and man game in the presence of a circular obstacle," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2009, pp. 5045–5050.

[26] R. Murrieta-Cid, T. Muppirala, A. Sarmiento, S. Bhattacharya, and S. Hutchinson, "Surveillance strategies for a pursuer with finite sensor range," *The International Journal of Robotics Research*, vol. 26, no. 3, pp. 233–253, 2007.

[27] S. D. Bopardikar, F. Bullo, and J. P. Hespanha, "On discrete-time pursuit-evasion games with sensing limitations," *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1429–1439, 2008.

[28] L. E. Parker, "Distributed algorithms for multi-robot observation of multiple moving targets," *Autonomous Robots*, vol. 12, no. 3, pp. 231–255, 2002.

[29] B. Jung and G. S. Sukhatme, "Tracking targets using multiple robots: The effect of environment occlusion," *Autonomous Robots*, vol. 13, pp. 191–205, 2002.

[30] M. Zhang and S. Bhattacharya, "Multi-agent visibility based target tracking game," in *Distributed Autonomous Robotic Systems*, ser. Springer Tracts in Advanced Robotics. Springer, 2016, pp. 271–284.

[31] B. Tovar and S. M. LaValle, "Visibility-based pursuit-evasion with bounded speed," *The International Journal of Robotics Research*, vol. 27, no. 11–12, pp. 1350–1360, 2008.

[32] H. H. González-Baños, C.-Y. Lee, and J.-C. Latombe, "Real-time combinatorial tracking of a target moving unpredictably among obstacles," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2002, pp. 1683–1690.

[33] T. Bandyopadhyay, Y. Li, M. H. Ang, and D. Hsu, "A greedy strategy for tracking a locally predictable target among obstacles," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2006, pp. 2342–2347.

[34] B. Penin, P. R. Giordano, and F. Chaumette, "Vision-based reactive planning for aggressive target tracking while avoiding collisions and occlusions," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 3725–3732, 2018.

[35] B. F. Jeon and H. J. Kim, "Online trajectory generation of a MAV for chasing a moving target in 3D dense environments," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 1115–1121.

[36] R. Bonatti, W. Wang, C. Ho, A. Ahuja, M. Gschwindt, E. Camci, E. Kayacan, S. Choudhury, and S. Scherer, "Autonomous aerial cinematography in unstructured environments with learned artistic decision-making," *Journal of Field Robotics*, vol. 37, no. 4, pp. 606–641, 2020.

[37] S. Engin, Q. Jiang, and V. Isler, "Learning to play pursuit-evasion with visibility constraints," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2021, pp. 3858–3863.

[38] V. Chvátal, "A combinatorial theorem in plane geometry," *Journal of Combinatorial Theory, Series B*, vol. 18, pp. 39–41, 1975.

[39] S. Fisk, "A short proof of Chvátal's watchman theorem," *Journal of Combinatorial Theory, Series B*, vol. 24, no. 3, p. 374, 1978.

[40] J. O'Rourke, *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.

[41] D. T. Lee and A. K. Lin, "Computational complexity of art gallery problems," *IEEE Transactions on Information Theory*, vol. 32, no. 2, pp. 276–282, 1986.
[42] M. Abrahamsen, A. Adamaszek, and T. Miltzow, "The art gallery problem is $\exists\mathbb{R}$-complete," *Journal of the ACM*, vol. 69, no. 1, pp. 4:1–4:70, 2022.
[43] D. C. Tozoni, P. J. de Rezende, and C. C. de Souza, "Algorithm 966: A practical iterative algorithm for the art gallery problem using integer linear programming," *ACM Transactions on Mathematical Software*, vol. 43, no. 2, pp. 16:1–16:27, 2016.
[44] P. J. de Rezende, C. C. de Souza, S. Friedrichs, M. Hemmer, A. Kröller, and D. C. Tozoni, "Engineering art galleries," in *Algorithm Engineering*, ser. Lecture Notes in Computer Science. Springer, 2016, vol. 9220, pp. 379–417.
[45] D. T. Lee and F. P. Preparata, "An optimal algorithm for finding the kernel of a polygon," *Journal of the ACM*, vol. 26, no. 3, pp. 415–421, 1979.
[46] J. O'Rourke and K. J. Supowit, "Some NP-hard polygon decomposition problems," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 181–190, 1983.
[47] J. M. Keil, "Polygon decomposition," in *Handbook of Computational Geometry*, J.-R. Sack and J. Urrutia, Eds. Elsevier, 2000, pp. 491–518.
[48] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," in *IEEE Trans. Robotics and Automation*, vol. 12, no. 4, 1996, pp. 566–580.
[49] T. Siméon, J.-P. Laumond, and C. Nissoux, "Visibility-based probabilistic roadmaps for motion planning," *Advanced Robotics*, vol. 14, no. 6, pp. 477–493, 2000.
[50] H. Choset and J. Burdick, "Sensor-based exploration: The hierarchical generalized Voronoi graph," *The International Journal of Robotics Research*, vol. 19, no. 2, pp. 96–125, 2000.
[51] D. Portugal and R. Rocha, "A survey on multi-robot patrolling algorithms," *Lecture Notes in Electrical Engineering*, vol. 89, pp. 139–146, 2011.
[52] N. Basilico, N. Gatti, and F. Amigoni, "Patrolling security games: Definition and algorithms for solving large instances with single patroller and single intruder," *Artificial Intelligence*, vol. 184–185, pp. 78–123, 2012.
[53] S. L. Smith, M. Schwager, and D. Rus, "Persistent robotic tasks: Monitoring and sweeping in changing environments," *IEEE Transactions on Robotics*, vol. 28, no. 2, pp. 410–426, 2012.
[54] S. Alamdari, E. Fata, and S. L. Smith, "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations," *The International Journal of Robotics Research*, vol. 33, no. 1, pp. 138–154, 2014.
[55] V. Isler, D. Sun, and S. Sastry, "Roadmap based pursuit-evasion and collision avoidance," in *Proc. Robotics: Science and Systems (RSS)*, Cambridge, MA, USA, 2005.
[56] I. Becerra, R. Murrieta-Cid, R. Monroy, S. Hutchinson, and J.-P. Laumond, "Maintaining strong mutual visibility of an evader moving over the reduced visibility graph," *Autonomous Robots*, vol. 40, no. 2, pp. 395–423, 2016.
[57] G. J. Laguna and S. Bhattacharya, "Tracking mobile intruders in an art gallery: Guard deployment strategies, fundamental limitations, and performance guarantees," arXiv:1807.08350, 2018. [Online]. Available: https://arxiv.org/abs/1807.08350
[58] G. J. Laguna and S. Bhattacharya, "Path planning with incremental roadmap update for visibility-based target tracking," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 1159–1164.
[59] G. Laguna, S. Mandal, and S. Bhattacharya, "Roadmap for visibility-based target tracking: Iterative construction and motion strategy," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2021, pp. 4732–4737.
[60] S. Mandal and S. Bhattacharya, "Relay pursuit for multirobot target tracking on tile graphs," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2023, pp. 7691–7698.
[61] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Communications of the ACM*, vol. 22, no. 10, pp. 560–570, 1979.
[62] I. Becerra, R. Murrieta-Cid, and R. Monroy, "Evader surveillance under incomplete information," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2010, pp. 5511–5518.
[63] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum Press, 1972, pp. 85–103.
[64] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
[65] V. Chvátal, "A greedy heuristic for the set-covering problem," *Mathematics of Operations Research*, vol. 4, no. 3, pp. 233–235, 1979.
[66] S. L. Hakimi, "Optimum locations of switching centers and the absolute centers and medians of a graph," *Operations Research*, vol. 12, no. 3, pp. 450–459, 1964.
[67] O. Kariv and S. L. Hakimi, "An algorithmic approach to network location problems. I: The p-centers," *SIAM Journal on Applied Mathematics*, vol. 37, no. 3, pp. 513–538, 1979.
[68] H. Edelsbrunner, *Algorithms in Combinatorial Geometry*. Springer, 1987.
[69] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1495–1512, 2013.
[70] S. Gillies *et al.*, "Shapely: manipulation and analysis of geometric objects," https://github.com/shapely/shapely, 2007.
[71] K. J. Obermeyer *et al.*, "VisiLibity: a 2D exact visibility library," https://karlobermeyer.github.io/VisiLibity1/, 2008.